MDPI

Article

LLM-based cyberattacks detection using network flow statistics

Leopoldo Gutiérrez-Galeano[1]*, Juan-José Domínguez-Jiménez[1], Jörg Schäfer[2] and Inmaculada Medina-Bulo[1]

[1] Escuela Superior de Ingeniería, Universidad de Cádiz, Avda. Universidad de Cádiz, 10, Puerto Real, 11519, Spain; juanjose.dominguez@uca.es (J.D.); inmaculada.medina@uca.es (I.M.)
[2] Faculty of Computer Science and Engineering, Frankfurt University of Applied Sciences, Nibelungenplatz 1, Frankfurt am Main, 60318, Germany; jschaefer@fb2.fra-uas.de
* Correspondence: leopoldo.gutierrez@uca.es

Abstract: Cybersecurity is a growing area due to the new types of cyberthreats that are constantly emerging. Tools and techniques exist to keep systems secure against certain known types of cyberattacks, but they are not sufficient for others that have recently emerged. Therefore, research is needed to design new strategies to deal with new types of cyberattacks as they emerge. Existing tools that use Artificial Intelligence techniques mainly use Artificial Neural Networks designed from scratch. In this paper, we show results of a new approach using an encoder-decoder pre-trained Large Language Model and, using Fine-Tuning, adapt its classification scheme for the detection of cyberattacks. Our system is anomaly-based and takes statistics of already finished network flows as input. We carried out the experiments applying undersampling and using a Stratified 5-Fold Cross-Validation approach to the CSE-CIC-IDS2018, CIC-IDS-2017 and BCCC-CIC-IDS-2017 datasets, to check if our approach is valid regardless of the dataset used. The results show an accuracy, precision, recall and F-score of more than 99.84%, 99.94% and 99.90%, respectively, for the detection of cyberattacks.

Keywords: Large Language Model; Machine Learning; Fine-Tuning; Deep Learning; Cybersecurity; Network Security; Cyber-threat; Attacks Detection; Multi-class classification; Attack classification

1. Introduction

Cybersecurity is a necessary area due to new risks that are constantly appearing. Considering the wide range of threats that can affect a computer system, it is necessary to make use of several techniques and tools to minimise the impact of constant attacks that occur on computer networks. However, this is still insufficient, as cybercriminals are always ahead of the curve and continuously develop new types of cyberthreats [1].

Therefore, it is necessary to protect information since it is the most important asset of an organisation. To this end, we have a wide range of tools at our disposal that can be used to reduce the risks faced by these systems.

In terms of protecting organisation networks, we have available the so-called *Security Information and Event Management* systems (SIEM) [2], *Intrusion Prevention Systems* (IPS) [3], and *Intrusion Detection Systems* (IDS) [4].

An IDS can be classified in *host-based* and *network-based*. The first monitors a computer and the second one, known as *Network Intrusion Detection Systems* (NIDS), monitors a network. Finally, a NIDS could be classified as: (a) *signature-based*, focused on signatures of attacks or specific patterns; or (b) *anomaly-based*, which is focused on abnormalities,

Received:
Revised:
Accepted:
Published:

Citation: Gutiérrez-Galeano, L.; Domínguez-Jiménez, J.J.; Schäfer, J.; Medina-Bulo, I. LLM-based cyberattacks detection using network flow statistics. *Appl. Sci.* **2025**, *1*, 0. https://doi.org/

malicious activity, knowing the normal behaviour of computer networks [5]. We would like to point out that our approach is aligned to *anomaly-based* NIDS.

The main purpose of the aforementioned tools is the detection of cyberattacks. However, the continuous appearance of new types of attacks makes it necessary to have cyberattack detection systems that can infer new attacks by using *Artificial Intelligence* (AI) techniques. For the development of this type of systems, we could use *Deep Learning* (DL) [6] algorithms and, more specifically, a model based on *Artificial Neural Networks* (ANN) [7].

This task could be solved by defining and training a model from scratch, or by selecting an existing one already pre-trained, to apply *Fine-Tuning* or *Transfer Learning*, which, although they are similar techniques, maintain their differences. The use of pre-trained models is rising if we are interested in building a new model, usually focused on solving a particular task related to the original one. This approach is widely used in supervised learning for models based on different tasks, such as face recognition, object recognition, *Natural Language Processing* (NLP), and so on [8].

Transfer Learning [9] is a well-established technique focused on modifying part of the architecture of an existing ANN that has already been trained, replacing the last layers with new ones that are relevant to the new task, and then retraining only the altered layers to adapt a model to a new domain.

In contrast, *Fine-Tuning* [10] is an approach used to modify weights of an existing ANN, without altering its architecture, to solve a specific task, different from the one for which it was initially trained. The starting point is an already pre-trained model, which will be transformed into a new one after applying a training step using new data. *Fine-Tuning* techniques are typically used to solve a more specific issue, within the purpose for which a model was initially trained. However, in our case, we radically change the purpose for which the model was initially trained. Although by applying this technique, all layers of a model require some learning, this is faster than training a model from scratch [9].

Section 4 contains a bibliographic study with several publications on the detection of attacks using AI techniques, most of them based on DL, such as [11–24]. As a subset of DL, Large Language Models (LLM) have evolved from their roots in NLP to encompass a wider range of applications, including cybersecurity. Following the classification proposed in Liu et al. [25], LLMs can be classified into three types based on their transformer architecture: decoder-only, encoder-only, and encoder-decoder models. Decoder-only models, such as GPT-3, excel at generating text, while encoder-only models, such as BERT, are better suited for tasks like text classification. Encoder-decoder models, such as *T5* [26], combine the strengths of both approaches and can perform a wide range of tasks. However, we just found two publications on cyberattack detection models based on *Fine-Tuning* LLM. Li et al. [27] used an encoder-only model, such as BERT, whereas Manocchio et al. [28] compared the use of an encoder-only model and a decoder-only model, such as BERT and GPT 2.0, respectively. We present a new approach for the detection of cyberattacks, based on the *Fine-Tuning* of an encoder-decoder LLM pre-trained for NLP tasks, more concretely *T5*. This strategy allows us to evaluate the most suitable LLM based architecture for the detection of cyberattacks. The experiments performed obtained promising results for the selected metrics, *accuracy*, *precision*, *recall* and *F-score* [29], for the aforementioned task, using the smallest available size of the *T5* model. The results are 99.84%, 99.94% and 99.90%, respectively, for all metrics, applying our method to the CSE-CIC-IDS2018, CIC-IDS-2017 and BCCC-CIC-IDS-2017 datasets. Taking into account all the results of the papers found in the bibliographic study, as can be seen in Section 4, our results are better.

The present paper describes a cyberattack detection system using a DL model. More concretely, we carry out a *Fine-Tuning* of an encoder-decoder model, for NLP, to obtain

a new model focused on solving a completely different task, which is the detection of cyberattacks. We sincerely believe that our strategy is a novel approach as we did not find any paper based on transformer-based encoder-decoder models. Anyway, after comparing our results with other LLM based strategies, we can claim that our system achieves better results.

The structure of the paper is as follows. Section 2 details the most relevant datasets for cyberattacks detection, introduces the design of our system, describes the data preprocessing process, including data cleaning and data transformation, analyses the data cleaning results in depth, undersampling, data splitting, and describes the *Fine-Tuning* process; Section 3 details the results for the performed experiments; Section 4 describes the papers found regarding strategies based on LLM, *Machine Learning* (ML) in general and finally compares our results with the found papers together, paying attention to each dataset; and finally, Section 5 presents the conclusions and the future work to be done.

2. Materials and Methods

2.1. Cyberattacks datasets

The number of public datasets related to IDS is limited, even more if we are looking for a good dataset. The following datasets are the most relevant:

KDD Cup 1999 [30] was published by the University of California, Irvine in 1999. This dataset is an updated version of DARPA98. It has been widely used in academic environment, for the research about IDS.

NSL-KDD [31] was published in 2009, and this is the improved and updated version of the KDD Cup 1999 dataset. Basically, it solves different problems that were detected in the previous version.

UNSW-NB15 [32] has been published by the University of New South Wales (UNSW) for academic purposes. The training set has been generated over a period of time of 16 hours and the test set on a period of 15 hours. This dataset contains 9 types of attacks and 49 features.

CIC-IDS-2017 [33,34] appears due to the lack of adequate datasets to properly evaluate the new cyberattack detection models that researchers have been developing. It was published by the Canadian Institute for Cybersecurity (CIC). They concluded that all the public datasets published before are unreliable, outdated, and with little data volume. To fill this gap, they published a dataset that contains a wealth of information, including benign traffic and 14 types of attacks, grouped into 7 main types, which are the most popular so far. Moreover, they designed this dataset in an environment as realistic as possible. This dataset is still widely used because it still contains the most popular and up-to-date types of attacks. It was built using the CICFlowMeter [35] tool, which generates 84 features of network flow statistics.

CSE-CIC-IDS2018 [34,36] was published after a collaboration between CIC and the Communications Security Establishment (CSE). It follows the same structure as the previous dataset, CIC-IDS-2017. Although the main difference between both of them is that the latter version was configured over an Amazon Web Service (AWS) environment, both datasets are considered as the most modern and up-to-date datasets for IDS and they are still widely used. It was also built using the CICFlowMeter tool. Therefore, the number of features and its nature are the same. However, their creators selected almost the same types of attacks.

CIC-DDoS2019 [37,38] is another well-known dataset, published by the same entity, CIC, with a taxonomy of *DDoS* attacks.

AWID2 [39] and AWID3 [40] are datasets focused on *802.11* networks. They were published in 2016 and 2021, respectively. AWID2 contains the most popular attacks on *802.11* and is focused on their signatures. This dataset contains normal and attack traffic against *802.11* networks [41]. AWID3 is a newer version that includes some modern attacks, focused on *WPA2 Enterprise*, *802.11w* and *Wi-Fi 5*. It also contains multilayer attacks, such as *Krack and Kr00k* [42].

BCCC-CIC-IDS-2017 [43,44] , launched in 2024, this is an augmented dataset, published by the Behaviour-Centric Cybersecurity Center (BCCC), York University. They created this dataset processing the original PCAP files from the CIC-IDS-2017 dataset with NTLFlowLyzer [43,45], which is a data analyser developed by them that solves the weaknesses of CICFlowMeter. Unlike CIC-IDS-2017 and CSE-CIC-IDS2018, this dataset contains 122 features.

Table 1 contains a comparison among the datasets mentioned above. NSL-KDD includes the most types of attacks but has the lowest number of records. The one with the highest number of records is AWID2, but it is focused on *802.11* networks and contains only 3 attacks. CIC-IDS-2017 and CSE-CIC-IDS2018 include a high number of types of attacks, and specially the latter contains a high number of rows. In addition, CIC-IDS-2017 and CSE-CIC-IDS2018 [36] are considered the datasets that contain the most modern and up-to-date types of attacks. Both were built using the CICFlowMeter tool. Therefore, they present the same structure and contain 84 columns. Moreover, among their labels, both include 14 types of attacks, as well as the benign class. In Section 4, we list the most recent studies using CSE-CIC-IDS2018 for the detection of cyberattacks.

Table 1. Datasets for detection of cyberattacks

Dataset	Year	Attack Types	Features	Records
KDD Cup 1999	1999	4	41	4,898,431
NSL-KDD	2009	22	41	125,973
UNSW-NB15	2015	9	49	2,540,047
CIC-IDS-2017	2017	14	84	2,830,743
CSE-CIC-IDS2018	2018	14	84	16,232,943
CIC-DDoS2019	2019	18	84	12,794,627
AWID2	2016	3	155	42,388,298
AWID3	2021	13	253	15,574,911
BCCC-CIC-IDS-2017	2025	13	122	2,438,052

2.2. Designing the cyberattack detection system

Figure 1 shows the scheme of our proposed system for the detection of cyberattacks using AI techniques. First, network traffic is transformed by the *Extract, Transform, and Load* (ETL) module, which transforms network packets into network flow statistics. This module is implemented using CICFlowMeter [35]. CICFlowMeter is a network traffic flow generator that produces 84 network traffic characteristics [46]. After that, a preprocessing is performed by homogenising column names and removing useless columns and rows. Useless columns include those with missing values for most records, as well as columns with unique values and columns with high correlation. This preprocessing is described in more detail in Section 2.2.1.

Our proposal for the construction of a cyberattack detection system is the use of an existing pre-trained LLM. We applied a *Fine-Tuning* process to the selected model, *T5*, to

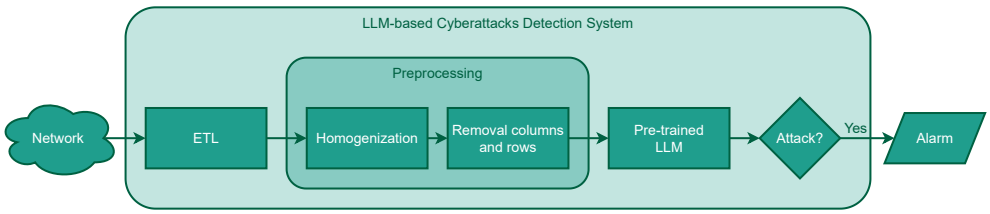

Figure 1. LLM-based Cyberattacks Detection System

make the complete adjustment of all their weights. Therefore, we changed its purpose from NLP tasks to cyberattack detection.

The *T5* model has a transformer-based architecture. It was developed by Google AI and was published in 2019. It is an *encoder-decoder* model, i.e. it uses *Recurrent Neural Networks* for *sequence-by-sequence* problem prediction, and it is prepared to solve text-to-text tasks. This type of architecture was relatively new and, in fact, was included in Google's translation service in 2016 [47].

Finally, the output of the model points out the existence or not of an attack, to produce the corresponding alarm related to an existing cyberattack.

In order to train the mentioned cyberattack detection model, we first needed to select a sufficiently good dataset, CSE-CIC-IDS2018, which contains statistics of network flows of a variety of the most up-to-date attack types. In this case, we did not need to perform the transformation with any ETL since the dataset included the required information.

Despite the fact that this dataset was released more than 6 years ago, it is still useful to build real systems since it is the most up-to-date dataset, with the most modern and popular types of attacks and with quite a lot of information, as described in Section 2.1. All the mentioned attributes make the selected dataset the most suitable and useful for building a good enough system.

We preprocessed the aforementioned dataset to optimise the training of our model, applying undersampling to the benign data in order to balance benign and malignant data, generating different training and validation subsets, as well as a test subset. Using the generated pairs of training and validation, we fine-tuned the selected pre-trained model. Finally, the adjusted model was evaluated with the previously generated test subset. The final model predicts the specific type of attack given statistics of networks flows.

Figure 2 shows this process, where the *T5* model is used as the basis of our system and the whole process starts preprocessing CSE-CIC-IDS2018.

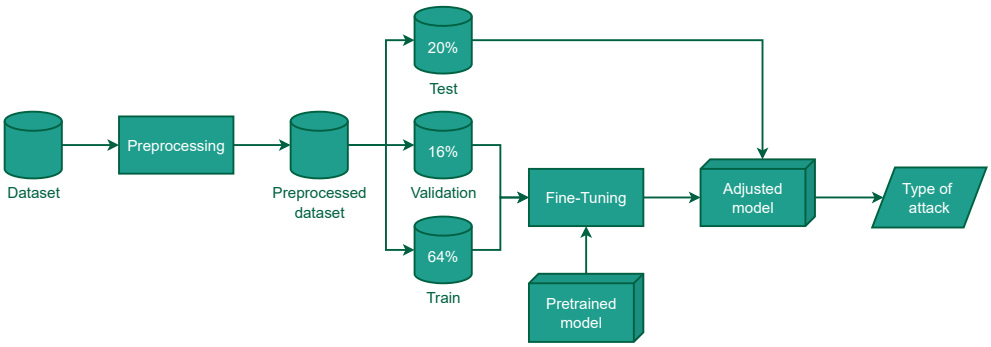

Figure 2. Training scheme of the system

2.2.1. Data Preprocessing

Data preprocessing is extremely important for the experiments to be carried out successfully. The main purposes of this task are: (a) filtering useless data by removing

columns and rows which do not provide extra information to the model or could cause worse results, and (b) the data adaption in order to be used directly in experiments.

Data cleaning In the following paragraphs can be found the detailed data cleaning process:

- Homogenisation of column names: The selected dataset contains column names with all lowercase letters, words with first letter capitalised, words with all letters capitalised, words separated by blanks or with the underscore character, and most names start with a blank space. We agreed to remove initial blanks as well as homogenise column names. To do this, all leading and trailing blanks were removed, all blanks were replaced by an underscore, and all letters were changed to upper case.
- Removal of useless columns: In the selected dataset you can find some useless columns, since they are available for just a few rows of data, located in just one file. This fact means missing values for most records. These columns are: (a) FLOW_ID identifies the flow, (b) SRC_IP identifies the source machine by its IP address, (c) SRC_PORT identifies the port in the source machine, and (d) DST_IP identifies the destination machine by its IP address. With the exception of the columns mentioned above, which were removed, the rest of the columns have been kept.
- Removal of columns with unique values: Columns with unique values are not useful for model building as they do not provide extra information. To carry out this process, the *standard deviation* of all columns is calculated, and if the deviation is 0, it means that all values are equal. If the *deviation* is close to 0, it means that almost all values are equal, and, in fact, it would also not contribute anything to the future model. In the experiments, we compared the number of columns with a *standard deviation* of 0 and with a very low *standard deviation*, for example 0.01, and applying both filters, the result was that we obtained the same columns to be removed. Hence, we decided to eliminate all columns with a *standard deviation* equal to 0, since for both filters we would have to remove the same columns.
- Removal of columns with high correlation: Columns with high correlation were removed because they are not useful for the future model. The steps are as follows: (a) calculate the *correlation matrix*, (b) obtain the *upper triangular matrix*, (c) list the columns whose correlation is greater than 0.95, and (d) remove the columns with high correlation.
- Removal of infinite, empty and null values: Infinite, empty, and null values are useless, since they do not provide additional information to the model. Therefore, rows containing such values have been deleted.
- Removal of duplicate rows: Finally, duplicated rows have to be removed, since they do not contribute anything to our dataset, as a common task in a data cleaning process.

Data transformation The features contained in CSE-CIC-IDS2018, after cleaning, cannot be used directly with the *T5* model, due to its architecture. Therefore, we need to apply a couple of transformations:

- Preparation of input strings: The *T5* model requires a string of characters as input. We cannot forget that this model has been trained to solve tasks related to NLP. Thus, for each row of data, a single input string with a unified syntax is required, given each input feature. To address the challenge of multiple input features, we adopted a data representation strategy similar to Gutierrez-Galeano

[39]. They used the selected model, T5, to translate English sentences into French, including jokes and puns. The issue was similar, that is, more than one input feature against a single input string in the model. In that case, the vertical bar | was used as a separator between the input features. To prepare the CicFlowMeter data for the LLM, we performed a custom token concatenation process. Each feature was converted into a token, and these tokens were concatenated into a sequence, separated by the | delimiter. This sequence was then fed into the LLM. This approach allowed us to represent the complex network flow data in a format that the LLM could easily process, enabling us to extract meaningful insights and patterns. Figure 3 shows an example of the construction of input strings, given statistics of network flows. At the top are located the characteristics and at the bottom is the separator character. The box in the middle part contains an already prepared entry. The arrows indicate where each value would be set, together with the vertical bar between them.

- Labelling: The labels with the types of attacks contained in the dataset are strings with alphanumeric characters, and, in this way, they are useful if we are interested in identifying the type of attack at first sight. However, labels must be encoded to avoid issues in classification. We decided to encode the labels using sequential numbers, starting from 0. Table 5 contains the string labels with the types of attacks and the corresponding number label assigned to the selected dataset.

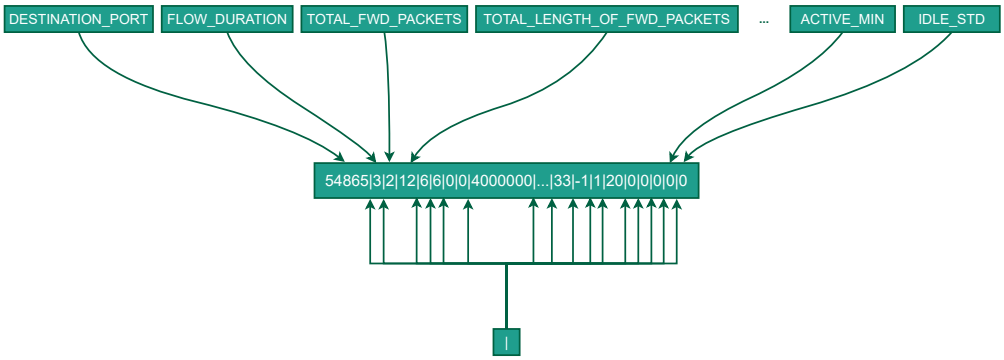

Figure 3. Concatenation example

2.2.2. Data cleaning results in depth

The initial dataset consists of 16,232,943 rows and 84 features. After cleaning, in total, we have 15,705,343 rows and 45 columns, which were used to build our system. Table 2 contains a quick summary of the data cleaning results.

Table 3 contains a complete analysis after applying the whole data cleaning process to the selected dataset. The most interesting values are in bold. We would like to point out some interesting facts. The labels *SQL Injection*, *Brute Force-XSS*, *Brute Force-Web* and *DDOS attack-LOIC-UDP* did not lose any data row. In addition, the most affected types were *DoS attacks-SlowHTTPTest* and *FTP-BruteForce*, since we removed 86.09% and 79.65% of the rows, respectively. Furthermore, the proportions among the attack types are very different.

Table 2. Summarization of data cleaning results

Step	Rows	Removed rows	Columns	Removed columns
Initial status	16,232,943		84	
Removal of useless columns	16,232,943		80	4
Removal of columns with unique values	16,232,943		72	8
Removal of columns with high correlation	16,232,943		45	27
Removal of infinite, empty and null values	16,137,183	95,760	45	
Removal of duplicate rows	15,705,343	431,840	45	

Table 3. Statistics of the data cleaning process by type of attack

Types of attacks	Initial rows	After cleaning	Removed rows	Removed proportion
Benign	**13,484,708**	**13,352,392**	**132,316**	**0.98 %**
DDoS attacks-LOIC-HTTP	576,191	576,175	16	0.00 %
DDOS attack-HOIC	686,012	668,461	17,551	2.56 %
DoS attacks-Hulk	461,912	434,873	27,039	5.85 %
Bot	286,191	282,310	3,881	1.36 %
Infilteration	161,934	160,604	1,330	0.82 %
SSH-Bruteforce	187,589	117,322	70,267	37.46 %
DoS attacks-GoldenEye	41,508	41,455	53	0.13 %
DoS attacks-Slowloris	10,990	10,285	705	6.41 %
DDOS attack-LOIC-UDP	1,730	1,730	**0**	**0 %**
Brute Force -Web	611	611	**0**	**0 %**
Brute Force -XSS	230	230	**0**	**0 %**
SQL Injection	**87**	**87**	**0**	**0 %**
DoS attacks-SlowHTTPTest	139,890	19,462	**120,428**	**86.09 %**
FTP-BruteForce	193,360	39,346	**154,014**	**79.65 %**
	16,232,943	15,705,343	527,600	3.25 %

Table 4. Statistics of the data cleaning process grouping by benign/malignant

Types of attacks	Initial rows	After cleaning	Removed rows	Removed proportion
Benign	**13,484,708**	**13,352,392**	**132,316**	**0.98 %**
Malignant	**2,748,235**	**2,352,951**	**395,284**	**14.38 %**
	16,232,943	15,705,343	527,600	3.25 %

2.2.3. Undersampling

Table 4 contains an analysis similar to that performed for Table 3 but classifies the dataset into two types of network flows, benign and malignant. The original dataset exhibited a significant class imbalance because the number of benign instances far exceeded the number of malignant ones. Originally, the dataset contained 13,484,708 benign rows and 2,748,235 malignant rows, 83.07% and 16.93%, respectively. The data cleaning process removed a substantial number of rows, but disproportionately affected the benign class. This further exacerbated the existing class imbalance. Even after cleaning, the dataset remained highly imbalanced, with 13,352,392 benign rows and 2,352,951 malignant rows. We would like to point out the proportion of data removed for both benign and malignant types of attacks. The proportion of data removed is 0.98% benign and 14.38% malignant, which is 132,316 and 395,284 rows, respectively.

Class imbalance can significantly impact the performance of machine learning models. Models trained on imbalanced datasets may become biased towards the majority class, leading to poor performance on the minority class. To address the severe class imbalance, the authors applied the undersampling technique. Undersampling is a common technique for addressing class imbalance. It involves reducing the number of instances in the majority class to match the number of instances in the minority class. Therefore, we agreed to balance the dataset with a proportion of approximately 50% benign data and 50% malignant data. Therefore, we decided to randomly select 20% benign rows, resulting in a more balanced dataset, in terms of proportions between classes, and a balanced dataset in terms of proportions between benign and malignant data.

Table 5 shows the final number of rows and their proportion for each class, after cleaning and after performing an undersampling of 20% benign network flows. The final proportions are slightly more balanced, since there are 53.16% benign rows, and the proportions of malignant labels increased by 313%. Therefore, after undersampling, the malignant types with the most rows have 13.31%, 11.47% and 8.66%, for the classes *DDOS attack-HOIC*, *DDoS attacks-LOIC-HTTP* and *DoS attacks-Hulk*, respectively.

Table 6 contains a similar analysis but classifies the rows as benign and malignant. After cleaning, the dataset had 13,352,392 benign rows and 2,352,951 malignant rows, which is 85.02% and 14.98%, respectively. At this point, the dataset had a pronounced imbalance. Hence, after applying the aforementioned undersampling of 20% benign rows of data, the proportions approach 53.16% benign network flows and 46.84% are malignant.

The proportions are specified in Table 5 and Table 6 since the imbalance of the selected dataset. In addition to being useful for analysing the distribution of data for each type of attack, these values are also useful together with the confusion matrix in Table 8 to study the results, even to calculate the weighted metrics. We would like to highlight that in terms of rows per label, the dataset is still imbalanced, but slightly less than before, while in terms of proportions between benign and malignant network flows, the dataset is now balanced.

2.2.4. Data splitting

At this point, we have a preprocessed dataset with two columns: source_text with the prepared input features, just as explained in Section 2.2.1; and target_text with their labels. The only step left is the separation of the dataset into different subsets to fine-tune each model in the right way. This means that we need train and validation subsets together to train each model and, finally, we need the test subset, with data which each model has never seen, to obtain afterwards all values for each metric. The way we perform this task is splitting the preprocessed dataset using *Cross Validation* (CV).

The agreed proportions are as follows. After performing a data shuffle, we fix the first 80% to apply the *Stratified 5-Fold CV* and the remaining 20% is fixed as the test subset. This

Table 5. Dataset statistics by type of attack after applying undersampling

Types of attacks	After cleaning		Undersampling 20% Benign	
	Rows	Prop.	Rows	Prop.
Benign	13,352,392	85,02 %	2,670,478	53,16 %
DDoS attacks-LOIC-HTTP	576,175	3,67 %	576,175	11,47 %
DDOS attack-HOIC	668,461	4,26 %	668,461	13,31 %
DoS attacks-Hulk	434,873	2,77 %	434,873	8,66 %
Bot	282,310	1,80 %	282,310	5,62 %
Infilteration	160,604	1,02 %	160,604	3,20 %
SSH-Bruteforce	117,322	0,75 %	117,322	2,34 %
DoS attacks-GoldenEye	41,455	0,26 %	41,455	0,83 %
DoS attacks-Slowloris	10,285	0,07 %	10,285	0,20 %
DDOS attack-LOIC-UDP	1,730	0,01 %	1,730	0,03 %
Brute Force -Web	611	0,004 %	611	0,01 %
Brute Force -XSS	230	0,001 %	230	0,005 %
SQL Injection	87	0,001 %	87	0,002 %
DoS attacks-SlowHTTPTest	19,462	0,12 %	19,462	0,39 %
FTP-BruteForce	39,346	0,25 %	39,346	0,78 %
	15,705,343		5,023,429	

Table 6. Dataset statistics grouping by benign/malignant after applying undersampling

Types of attacks	After cleaning		Undersampling 20% Benign	
	Rows	Prop.	Rows	Prop.
Benign	**13,352,392**	**85.02 %**	**2,670,478**	**53.16 %**
Malignant	**2,352,951**	**14.98 %**	**2,352,951**	**46.84 %**
	15,705,343		5,023,429	

means that for all folds in the CV, the test subset always contains the same collection of data rows.

The *Stratified 5-fold CV* [48] is applied as follows. In the first fixed 80% of the data, we divided it into 5 parts, and for each fold, the *k-th* part is selected as the validation subset and the remaining 4 parts are selected as the training subset. That is, for the first fold, the first part, which contains the 16% of the data, is selected as the validation subset, and the other 4 parts, which contain the 64% of the data, is selected as the training subset. The remaining fixed 20% is considered as the test subset for all the folds.

In the case of the selected strategy, *Stratified* means that each subset contains the same proportion of data rows for each label. The selected dataset contains quite a lot of information. Therefore, in advance, this strategy could not be needed to be applied to the selected dataset. Nevertheless, keeping in mind the labels with very few rows of data, we agreed to apply it in order to obtain models which could classify these labels as good as possible.

2.2.5. Fine-tuning process

Training consists of adjusting the weights of each neuron of an ANN, so that predictions can be adjusted as closely as possible to the expected outputs. *Fine-Tuning* consists of using a pre-trained model as a starting point to solve a different problem. Basically, we load a pre-trained model and train it with our dataset, so that we adjust their weights to solve a new type of problem. Thus, instead of starting with a random weight configuration, we start with a weight configuration focused on solving a certain type of problem. In our case, the *T5* model has been pre-trained with the *C4* [26] dataset, "**C**olossal **C**lean **C**rawled **C**orpus" which was composed of hundreds of gigabytes of English text obtained from the internet, and after that we applied *Fine-Tuning* with another dataset, focused on an absolutely different scope, like the selected for our experiments.

Transfer Learning could be another option. This technique is similar to *Fine-Tuning* but there is a main difference between both. *Fine-Tuning* updates all the weights of an ANN, unlike *Transfer Learning*, in which several layers are locked, so that they keep their weights, and only the weights of the unlocked layers are adjusted. Therefore, we decided to adjust all the weights, since we are radically changing the purpose of the model, so our choice is a *Fine-Tuning*.

Once we decided to fine-tune an existing snapshot of the *T5* model, using the SimpleT5 [49] wrapper, the next step to carry out is the optimization of hyperparameters. SimpleT5 is a wrapper built over PyTorch, to perform *Fine-Tuning* of *T5* without any modification of its architecture. This package provides users with different functions to use different sizes of the *T5* model developing a few lines of code. It contains functions to downloading pre-trained snapshots and assigning them to a new instance, training models, predicting values or loading models previously trained with this package. SimpleT5 has several hyperparameters available to be used, which we carefully studied to select the most appropriate values for some of them and those that will be optimized. Once this task is completed, the fine-tuning of the model could be successfully carried out. Therefore, SimpleT5, provides users with the following hyperparameters:

- **max_epochs** is the maximum number of epochs for the training step. It has been set to 10, since LLMs have a high level of maturity and they do not need quite a lot of epochs to get a good model. Therefore, after performing the preliminary experiments, we realised that we never selected the best model of an experiment after the tenth epoch. Depending on the experiment, an overfitting could appear in one of the first epochs, even in the second one. Hence 10 is a value large enough.
- **target_max_token_len** contains the maximum number of output tokens. Since the selected model performs a classification task, it is not necessary to set a too large value. It should be pointed out that the output of the model is a numerical value between 0 and 14. The maximum number of tokens is 512, which is far from a necessary value. Preliminary experiments concluded that the minimum number of tokens is 3 and we observed that the values for the selected metrics keep the same trend for any value between 3 and 512. In addition, this parameter has the same behaviour than the variable source_max_token_len parameter in terms of the relationship between number of tokens and training time. Therefore, setting 3 as its value significantly reduces training times.
- **batch_size** is the parameter which contains the batch size that will be used to train this model. The default value is 8 and values power of 2 are usually used so, for instance, values like 8, 16 or 32 are usually used. The value 16 has been set, which is a medium value, since it is not the minimum and not too large either.
- **precision** is the parameter that contains the training precision. The available values are double precision (64), full precision (32) or half precision (16). By default, this

wrapper uses the value 32, which is the value used for the experiments. 32 has been selected because it is the central value, for instance, it is neither the smallest nor the largest value.

- **source_max_token_len** contains the maximum number of input tokens. The maximum number is 512 tokens. The higher the number of input tokens, the longer the training time, and therefore the lower the value, the shorter the training time. Logically, a shorter training time is desired, so it would be interesting to set this variable to the lowest possible value. However, given that the character strings used as input have an average of 189 characters, a minimum of 109 and a maximum of 345, it must be taken into account that if too few tokens are selected, then the model will not take into account much of the input information to carry out its task as satisfactorily as possible. Therefore, we must look for a balance, taking into account this parameter in the search for an optimal value that allows the model to be trained in the shortest possible time, while keeping the selected metrics as high as possible. We planned to perform the optimization with 50, 100, 150, 200, 250, 350 and 500 tokens. The evaluation of the hyperparameter optimization process has been carried out using the following evaluation metrics: *accuracy*, *precision*, *recall* and *F-score*.
  Table 7 shows all the results in depth. We would like to highlight the whole row for the value 150 of the maximum number of input tokens, since all values for all metrics for 150 are the best ones. The entire row has been formatted in bold. This fact does not mean that this is the best value, since all values for a greater maximum number of input tokens follow the same trend and differ slightly. Therefore, we added a new variable, which is the training time. We would like to highlight the increasing trend of this index, because the higher the number of input tokens, the longer the training time. Therefore, on the one side, all values for all metrics for the value 150 are enough stable to make the decision of selecting 150 as the best value for the aforementioned hyperparameter and, in fact, for this value we are getting the best results for the selected metrics. And, on the other hand, it is desirable the selection of a model which takes less time to be trained. Hence, we decided to select the minimum value among the aforementioned candidates, which is 150.

Table 7. Metrics for the hyperparameter optimization step

Max. Input Tokens	Accuracy	Precision	Recall	F-score	Training time
50	99.75 %	99.75 %	99.75 %	99.75 %	2:22:33
100	99.84 %	99.84 %	99.84 %	99.84 %	2:45:20
150	**99.84** %	**99.85** %	**99.84** %	**99.84** %	**3:15:47**
200	99.84 %	99.85 %	99.84 %	99.84 %	3:43:27
250	99.84 %	99.85 %	99.84 %	99.84 %	4:20:27
350	99.84 %	99.85 %	99.84 %	99.84 %	5:28:13
500	99.84 %	99.85 %	99.84 %	99.84 %	7:42:27

3. Experiment results

The experiments were performed in an environment with software that is commonly used for data science. We have selected Python to develop the experiments. All experiments were performed on the supercomputer of the University of Cádiz. It contains several NVIDIA A100 GPUs with 40GB of GPU memory, 256GB of RAM, each node with 2 CPUs,

each one with 64 cores. Moreover, it contains some nodes with more memory, more concretely with 1TB each node.

The hyperparameter optimization process led us to the selection of the best value for the maximum number of input tokens hyperparameter. Using the best configuration for the training step, we performed a *Fine-Tuning* task using the *Stratified 5-Fold CV* approach, which executes 5 different *Fine-Tuning* processes, each of them with its related pair of train and validation subsets. Each execution generates a snapshot that performs the classification between benign and malignant data and, in the second case, directly obtains the type of attack between the 14 types contained in the CSE-CIC-IDS2018 dataset. For each fold, since we configured the execution of 10 epochs, we obtained 10 model snapshots. Therefore, for each fold, we obtained the predictions using the validation subsets in order to select the best epoch, and finally, for the best one, we obtained the predictions using the test subset, which the model has never seen. We used these test predictions to calculate the values for each one of the selected metrics, for each fold, and to finish the whole process, we calculated the *mean* and the *standard deviation*, for each metric.

The evaluation of the experiments published in this paper, as well as the hyperparameter optimization process, has been carried out using the following evaluation metrics: *accuracy*, *precision*, *recall* and *F-score*. We would like to highlight that we have calculated the weighted metrics since the dataset is imbalanced, in terms of rows of data for each type of attack. In addition, these values allow us to compare our results with those published in other publications. Keeping in mind the aforementioned information, we obtained 99.84%, for all the selected metrics.

All values for the calculated metrics could not say anything under certain conditions. Anyway, these metrics hide some information that could be interpreted in a confusion matrix. Table 8 shows the confusion matrix for the first fold. The results are very optimistic since the higher values are in the main diagonal and the remaining values are 0 or close to 0. In addition to that, we could extract two interesting conclusions from this matrix. The first is the number of false negatives that appear when the model predicted *Infiltration*, label number 5, where the right answer is *Benign*, label number 0. This fact means that, for the model, both classes are confusing for a minimum proportion of data. The other interesting fact is the way the model classifies *SQL Injection* data rows. The test subset contains 17 rows of data for this class and the model was just right 23.53% of times. Even this model erroneously predicted more rows of data for the *Brute Force-XSS* class, 7 times, and for the *Benign* class, 5 times.

In a more deep analysis, keeping in mind DoS attack types, which are classes 1, 2, 3, 7, 8, 9 and 13, we can affirm that our model classifies successfully all, or almost all, network flows. Misclassified values are negligible.

Brute force types of attacks are labeled as classes 6, 10, 11 and 14. The results are slightly different. For classes 6 and 14, *SSH-Bruteforce* and *FTP-Brute Force*, respectively, there is no misclassification. For the types of attacks 10 and 11, which are *Brute Force-Web* and *Brute Force-XSS*, respectively, results are improvable. For these classes, there are 13.11% and 8.7% misclassification.

Finally, the other types of attacks, 4, 5 and 12, are *Bot*, *Infiltration* and *SQL Injection*, respectively. The results are close to 100% for the first two types of attacks, obtaining only a misclassification of 0.01% and 0.22%. For the *SQL Injection* type of attack 23.53% of the data have been successfully classified.

Table 8. Main experiment confusion matrix for the first fold.

Label	Benign	DDoS attacks-LOIC-HTTP	DDOS attack-HOIC	DoS attacks-Hulk	Bot	Infilteration	SSH-Bruteforce	DoS attacks-GoldenEye	DoS attacks-Slowloris	DDOS attack-LOIC-UDP	Brute Force -Web	Brute Force -XSS	SQL Injection	DoS attacks-SlowHTTPTest	FTP-BruteForce	
Class	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	Misclassified
0	532,635	19	0	0	14	1,420	5	0	0	0	3	0	0	0	0	0.27 %
1	4	115,231	0	0	0	0	0	0	0	0	0	0	0	0	0	0 %
2	0	0	133,692	0	0	0	0	0	0	0	0	0	0	0	0	0 %
3	0	0	0	86,975	0	0	0	0	0	0	0	0	0	0	0	0 %
4	5	0	0	0	56,457	0	0	0	0	0	0	0	0	0	0	0.01 %
5	72	0	0	0	0	32,049	0	0	0	0	0	0	0	0	0	0.22 %
6	0	0	0	0	0	0	23,464	0	0	0	0	0	0	0	0	0 %
7	1	0	0	0	0	0	0	8,290	0	0	0	0	0	0	0	0.01 %
8	0	0	0	0	0	0	0	0	2,057	0	0	0	0	0	0	0 %
9	0	0	0	0	0	0	0	0	0	346	0	0	0	0	0	0 %
10	7	0	0	0	0	0	0	0	0	0	106	7	2	0	0	13.11 %
11	3	0	0	0	0	0	0	0	0	0	0	42	1	0	0	8.7 %
12	5	0	0	0	0	0	0	0	0	0	1	7	4	0	0	76.47 %
13	0	0	0	0	0	0	0	0	0	0	0	0	0	3,892	0	0 %
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7,869	0 %

4. Related work

In order to have an overview of the published papers on cyberattack detection models based on DL techniques, the results of the analysis of those research papers are presented in the current section.

In recent years, ML techniques, and more specifically DL techniques, have been used for the detection of cyberattacks, given the increasing development of new attacks and the amount of data to analyse in order to detect them. In this sense, we can differentiate between strategies that made use of LLM systems by *Fine-Tuning* models, such as our proposal, and other strategies that used other ML techniques [50].

4.1. Strategies based on Natural Language Processing

In this paper, we present a new approach that is the detection of cyberattacks, based on the *Fine-Tuning* of an encoder-decoder model pre-trained for NLP tasks. Regarding our approach, we found only a few papers related to NIDS based on a LLM, using the CSE-CIC-IDS2018 dataset, presented in this section. Both papers are based on an encoder-only model and the other one on encoder-only and decoder-only models. Moreover, we include a couple of papers based on the same line of research, but using a different dataset.

Li et al. [27] presented *Conditional Generative Adversarial Network* (CGAN) used together with BERT, for intrusion detection. They showed results after testing their approach with the datasets NF-ToN-IoT-V2, NF-UNSW-NB15 and CSE-CIC-IDS2018. However, in the paper, we did not find all the original types of attacks for the latter dataset, which is the one we used for our research. Finally, they prepared a comparison between the three aforementioned datasets using different methods. For the CSE-CIC-IDS2018 dataset, using their proposed approach, they obtained 98.22% of *accuracy*, 98.25% of *precision*, 98.22% of *recall* and 98.23% of *F-score*. For the other methods, their *accuracies* are between 86.07% and 96.83%, *precision* rates between 85.91% and 97.46%, *recall* rates between 86.07% and 96.83%, and *F-score* between 84.39% and 97.01%.

Manocchio et al. [28] introduced *FlowTransformer*, which is a transformer framework for *flow-based* NIDS. They used three datasets for the study of results as CSE-CIC-IDS2018, NSL-KDD and UNSW_NB15. They used different approaches and two well-known transformer architectures, such as GPT 2.0 and BERT. For the CSE-CIC-IDS2018 dataset, using the GPT-based model, they obtained an *accuracy* of 95.86% and *F-score* of 96.93%, and for the BERT-based model they obtained an *accuracy* of 95.48% and *F-score* of 95.80%.

The references mentioned in the previous two paragraphs are the only ones working on an approach based on a LLM, for the detection on cyberattacks, using the same dataset that we are using. After making a comparison between both studies and our proposed approach, we can claim that our results are better than these, as we explain in Section 4.3.

Ferrag et al. [51] presented an architecture based on BERT, called *SecurityBERT*, they obtained 98.2% *accuracy* and 98% of *precision*, *recall* and *F-score*. They reported an inference time of 0.15 seconds on an average CPU and a compact model size of 16.7MB. They also published that it is ideal for deployment in IoT devices. In the state of the art they listed several papers related to BERT and cybersecurity in general. They used the Edge-IIoTset dataset that contains network traffic but it is focused on IoT.

Lira et al. [52] introduced a model called *BERTIDS*, based on BERT. This paper was published in 2024 but they used the NSL-KDD dataset, published in 2009. They published a comparative table with other approaches, which obtained *accuracies* of no more than 90%. They obtained 98.01% of *accuracy*, 98.31% of *precision*, 98.01% of *recall* and 98.09% of *F-score*. In this sense, it is not comparable to our results as they are different datasets and different models, but we can highlight the following: we used a more modern dataset (with the most up-to-date and popular types of attacks), our model is based on a more modern model (both are LLM, both created by Google, but BERT was published in 2018 and *T5* in 2019), and our *accuracy* is higher than 98.01%.

The results of the latter two papers are not directly comparable with our research, since they used different datasets.

4.2. Other Strategies based on Machine Learning

Given that there are not many research papers related to our approach, we are going to introduce some of the latest research results related to NIDS, which used different AI techniques. In order to carry out a related works analysis, we have focused on those strategies that make use of the selected dataset, to be able to compare the effectiveness of our proposal.

Macas et al. [5] published an interesting survey, which lists the techniques most used for each cybersecurity field. One of them is related to NIDS, and they introduced different AI techniques, but none of them used models based on NLP. However, they mentioned BERT, in addition to GPT-3, but for SPAM detection. This survey references two interesting papers: Abdel-Basset et al. [12] proposed a Semi-Supervised DL approach for Intrusion Detection (SS-Deep-ID) with the following results for CSE-CIC-IDS2018: 98.71% of *accuracy*,

94.91% of *precision*, 94.30% of *recall* and 94.92% of *F-score*. Their results were the following using the CIC-IDS-2017 dataset: 99.69 % of *accuracy*, 92.31 % of *precision*, 96.29 % of *recall* and 94.18 % of *F-score*. They compared their results with other papers and, between them, their proposal obtained the best results for the metrics they selected. In the other referenced paper, Nie et al. [13] proposed a *Generative Adversarial Network* (GAN)-based approach, using the CSE-CIC-IDS2018 dataset. They published a 95.32% of *accuracy*, 99.88% of *precision*, 95.85% of *recall* and 97.30% of *F-score*. They also compared their results with different approaches they found in different publications.

Wang et al. [22] proposed a Genetic Algorithm with Back Propagation Neural Network (GA-BPNN) model for computer network safety. They applied their approach to the CIC-IDS-2017 datasets. Their results were 98% of *accuracy*, 98.5% of *precision*, 95% of *recall* and 96.5% of *F-score*.

Maruthupandi et al. [11] introduced an Intelligent Attention Based Deep Convoluted Learning (IADCL) model. They applied their approach to different datasets, such as, NSL-KDD, CIC-IDS-2017 and CSE-CIC-IDS2018. And they compared their results with other approaches they applied, including Immune Net, XGB, Random Forest, Decision Tree and Logistic Regression. Keeping in mind the aforementioned techniques, that is, first of all their proposed approach and, after that, the other approaches, their results applying CSE-CIC-IDS2018 in terms of accuracy were 98.70%, 99.63%, 99.09%, 98.81%, 98.54% and 92.96%, precision of 99.72%, 99.64%, 99.03%, 98.82%, 93.41% and 98.80%, recall of 99.70%, 99.63%, 99.07%, 98.81%, 96.76% and 90.96%, and, F-score of 99.71%, 99.63%, 99.07%, 98.81%, 95.06% and 90.87%, respectively. And, about CIC-IDS-2017, accuracy of 99.70%, 99.63%, 99.09%, 98.81%, 98.54% and 92.96%, precision of 99.72%, 99.64%, 99.03%, 98.82%, 93.41% and 90.80%, recall of 99.70%, 99.63%, 99.07%, 98.81%, 96.76% and 90.96%, and, F-score of 99.71%, 99.63%, 99.07%, 98.81%, 95.06% and 90.87%, respectively.

Guo et al. [23] introduced a 1D-TCN-ResNet-BiGRU-Multi-Head Attention (TRBMA) based model, more concretely TRBMA (BS-OSS), a variant of the TRBMA model that integrates Borderline SMOTE-OSS hybrid sampling, applying their approach to the CIC-IDS-2017 dataset. Their results were 99.88% of *accuracy*, 99.86% of *precision*, 99.88% of *recall* and 99.89% of *F-score*.

Hariharan et al. [24] proposed a Hybrid Deep Learning Model for Network Intrusion Detection System Using Seq2Seq and ConvLSTM-Subnets, applied to the CIC-IDS-2017 dataset. Their results were 99% of *accuracy*, % of 97*precision*, % of 96*recall* and % of 97*F-score*.

Vo et al. [53] introduced an approach based on *Augmented Wasserstein GAN* (AWGAN) and *Parallel Ensemble Learning-based Intrusion Detection* (PELID) algorithms, known as APELID. They used the CSE-CIC-IDS2018 and NSL-KDD datasets for their study. Their results, after applying their approach with the first dataset, were 99.99% for all the metrics: *accuracy*, *precision*, *recall* and *F-Score*. This strategy is characterised by the fact that it balances the dataset using a GAN, generating synthetic data to improve training. This means that we cannot compare our results with those published this paper since we do not use the same data to train our model.

Amin et al. [54] proposed *Chaotic Zebra Optimization Algorithm and Long Short-Term Memory*, CZOLSTM, for the detection of cyberthreats. They used the CSE-CIC-IDS2018 dataset and their results were 99.92% of *accuracy* and 99.94% of *precision*, *recall* and *F-score*. They made a binary classification between benign and malignant data, and, in addition to taking just malignant data, they made a classification from a subset of types of attacks contained in the original dataset. Therefore, we cannot compare our results with theirs.

Kunang et al. [14] proposed a hybrid DL model for an IDS on the IoT platform. For this study, they used the Bot-IoT and CSE-CIC-IDS2018 datasets. For their proposal, and

using the latter dataset, their results were 99.17% of *accuracy*, 99.26% of *precision*, 99.17% of *recall* and 99.2% of *F-score*.

Bhardwaj et al. [15] proposed a framework based on a *Convolutional Neural Network* (CNN). They used different datasets as CSE-CIC-IDS2018, UNSW-NB15 and CIC-Darknet2020. For the first dataset, they obtained a 99.4% of *accuracy*, using their proposed framework.

Wang et al. [16] proposed an *Attention-based* CNN for IDS. They carried out their research using CSE-CIC-IDS2018. For their proposed model, they obtained 99.36% *accuracy*. They did not show results on other metrics.

Tapu et al. [17] proposed a novel idea of hybrid meta-DL to detect malicious packet data. They used a combination of *Siamese* and *Prototypical* networks, where the first was used for binary classification and the latter for multiclass classification. For their hybrid meta approach, using the CSE-CIC-IDS2018 dataset, their results were 90.64% of *accuracy*, 91.66% of *precision*, 90.64% of *recall* and 91% of *F-score*. And using CIC-IDS-2017, their results were 95.68% of *accuracy*, 96.50% of *precision*, 95.68% of *recall* and 96.10% of *F-score*

Li et al. [18] proposed the *Hierarchical and Dynamic Feature Extraction Framework* (HDFEF) for IDS. They showed results for different datasets as CIC-IDS-2017, UNSW-NB15 and CSE-CIC-IDS2018. For the first dataset, they obtained 99.73% of *precision*, 99.96% of *recall* and 99.84% of *F-score*. For the latter dataset, they obtained 98.22% of *precision*, 99.77% of *recall* and 98.49% of *F-score*. They did not show results about *accuracy*.

Yilmaz et al. [19] used different approaches for NIDS, including *Random Forest* (RF), *CatBoost*, *LightGBM* and *XGBoost*, applied to different datasets as CIC-IDS-2017, CSE-CIC-IDS2018 and INSDN. About their results with the CSE-CIC-IDS2018 dataset, using RF they obtained 93% of *precision*, 71% of *recall* and 74% of *F-score*, using *CatBoost* they obtained 89%, 83% and 83% respectively, using LightGBM they obtained 87%, 84% and 85% respectively and using *XGBoost*, they obtained 89%, 85% and 86% respectively. They did not show results related to *accuracy*. About CIC-IDS-2017, using RF they obtained 93% of *precision*, 71% of *recall* and 74% of *F-score*, using *CatBoost* they obtained 89%, 83% and 83% respectively, using LightGBM they obtained 87%, 84% and 85% respectively and using *XGBoost*, they obtained 89%, 85% and 86% respectively. They did not show results related to *accuracy*.

Hammad et al. [20] proposed a classifier using an approach called *Multinomial Mixture Modeling with Median Absolute Deviation and RF* (MMM-RF). They used the CSE-CIC-IDS2018 dataset and with their approach obtained 99.98% of *accuracy*. They do not show results related to other metrics.

Yu et al. [21] proposed a *Packet Bytes-based CNN* (PBCNN) for NIDS. They used CSE-CIC-IDS2018 and their results were 99.99% of *accuracy*, 98.2% of *precision* and 98.3% of *recall* and *F-score*.

4.3. Comparative analysis

We already described the effectiveness of our system, in terms of Accuracy, Precision, Recall and F-score, after applying our approach to the CSE-CIC-IDS2018 dataset. However, we decided to check if our methodology could be valid regardless of the dataset used. Therefore, we do not want to finish this study without showing results of our approach, using different datasets.

Table 9 contains all results together for all selected metrics, using the already studied CSE-CIC-IDS2018 as well as CIC-IDS-2017 and BCCC-CIC-IDS-2017. We obtained the best results applying our methodology to the CIC-IDS-2017 dataset. We obtained 99.94% for all the metrics, which is 0.1% better than applying it to CSE-CIC-IDS2018. Regarding the augmented dataset, we obtained 99.9% for all the metrics applying our approach to BCCC-CIC-IDS-2017. Keeping in mind that this dataset contains a larger set of features

available, we obtained results 0.04% worse than with the original dataset, CIC-IDS-2017. Anyway, we can see that all the results are quite similar and present promising values regarding the selected metrics for this paper.

Table 9. Results of our approach for different datasets.

DATASET	ACCURACY	PRECISION	RECALL	F-SCORE
CIC-IDS-2017	99.94 %	99.94 %	99.94 %	99.94 %
CSE-CIC-IDS2018	99.84 %	99.84 %	99.84 %	99.84 %
BCCC-CIC-IDS-2017	99.90 %	99.90 %	99.90 %	99.90 %

Therefore, at this point, we compare our results with the aforementioned research papers.

4.3.1. Using the CSE-CIC-IDS2018 dataset

Section 4.1 and Section 4.2 contain details on the publications included in this comparative. Table 10 contains the results published in these papers as well as ours, after applying our approach to the CSE-CIC-IDS2018 dataset. That table has been divided into three parts by dark horizontal lines. The first part, with only one row, contains the results of our approach.

The second part shows the results of two approaches based on LLM and the same dataset that we used. Our results are at least one percentage unit better than the other ones, for all metrics. Therefore, our approach is better among the aforementioned papers, and we would like to highlight that we selected the smallest size among the available snapshots of the *T5* model.

Finally, in the last part of the table mentioned above, we can find several references regarding IDS related models that used ML techniques, different from LLM, and the same dataset we used. Our results are again better. Keeping in mind that we selected the smallest available size of the *T5* model, we can claim that our results for all metrics are better than most of the papers found. Only two approaches, such as [20,21], obtained better *accuracies*, and just another one, which is [13], obtained slightly better *precision*. In case of Yu et al. [21], although *accuracy* is better, for the other metrics, their values are worse than the ours. In this sense, Harrell [55] affirms that *accuracy* can be a misleading metric for imbalanced datasets. Therefore, our system is better. Regarding Hammad et al. [20], although their *accuracy* is better, this fact does not say anything if we do not have information on any other metrics; hence we cannot affirm that their approaches are better than the ours.

Therefore, our approach is better among the LLM based research papers, and our results compared to the other ML based techniques are better for all the metrics, except for a couple of references, between which we obtained better results for the most of the selected metrics.

4.3.2. Using the CIC-IDS-2017 dataset

Section 4.2 contain details on the publications included in this comparative. Table 11 contains the results published in these papers as well as ours, after applying our approach to the CIC-IDS-2017 dataset. That table has been divided into two parts by dark horizontal lines. The first part, with only one row, contains the results of our approach.

The second part shows several references regarding IDS related models that used ML techniques, since we did not find any paper related to LLM, using CIC-IDS-2017. Our results are again better. Keeping in mind that we selected the smallest available size of the *T5* model, we can claim that our results for all metrics are better than almost all the papers found. Only one approach, Li et al. [25], obtained slightly better *recall*.

Table 10. Results for the selected metrics using CSE-CIC-IDS2018

Reference	Technique	Accuracy	Precision	Recall	F-score
Our approach	T5	**99.84 %**	**99.84 %**	**99.84 %**	**99.84 %**
Li et al. [27]	CGAN + BERT	98.22 %	98.25 %	98.22 %	98.23 %
Manocchio et al. [28]	GPT 2.0	95.86 %	–	–	96.93 %
	BERT	95.48 %	–	–	95.80 %
Maruthupandi et al. [11]	IADCL	99.82 %	99.80 %	99.81 %	99.80 %
	Immune Net	99.78 %	99.77 %	99.78 %	99.70 %
	XGB	99.00 %	99.03 %	99.00 %	99.01 %
	RF	98.81 %	98.82 %	98.81 %	98.81 %
	DT	98.69 %	93.41 %	98.79 %	96.03 %
	LR	87.96 %	90.80 %	87.96 %	88.99 %
Abdel-Basset et al. [12]	SS-Deep-ID	98.71 %	94.91 %	94.30 %	94.92 %
Nie et al. [13]	GAN	95.32 %	**99.88 %**	95.85 %	97.30 %
Kunang et al. [14]	Hybrid DL Model	99.17 %	99.26 %	99.17 %	99.20 %
Bhardwaj et al. [15]	CNN-1D	99.40 %	–	–	–
Wang et al. [16]	CNN	99.36 %	–	–	–
Tapu et al. [17]	Hybrid DL Model	90.64 %	91.66 %	90.64 %	91.00 %
Li et al. [18]	HDFEF	–	98.22 %	99.77 %	98.49 %
Yilmaz et al. [19]	RF	–	94.00 %	76.00 %	79.00 %
	CatBoost	–	91.00 %	80.00 %	83.00 %
	LightGBM	–	91.00 %	81.00 %	83.00 %
	XGBoost	–	89.00 %	83.00 %	84.00 %
Hammad et al. [20]	MMM-RF	**99.98 %**	–	–	–
Yu et al. [21]	PBCNN	**99.99 %**	98.20 %	98.30 %	98.30 %

Therefore, our approach is better among the ML based techniques.

4.3.3. Using the BCCC-CIC-IDS-2017 dataset

Finally, unfortunately, we did not find any research paper with results after applying the BCCC-CIC-IDS-2017 dataset, since it was launched while we were writing this paper.

5. Conclusions and future work

A cyberattack detection system has been designed. The results have been very promising, since we obtained 99.84%, 99.94% and 99.90%, using CSE-CIC-IDS2018, CIC-IDS-2017, and BCCC-CIC-IDS-2017, respectively, for all metrics, for a classification task between the benign profile and the 14 types of malignant network flows.

The main experiments for the construction of a model for the detection of cyberattacks have been designed and detailed. We built them using a *Fine-Tuning* approach, using as a starting point a pre-trained LLM for NLP tasks, more concretely *T5*, and a public dataset which contains statistics of network flows, based on the most popular and up-to-date types of attacks. It should be pointed out that, among all available pre-trained sizes, we selected the smallest, *t5-small*.

Table 11. Results for the selected metrics using CIC-IDS-2017

Reference	Technique	Accuracy	Precision	Recall	F-score
Our approach	T5	**99.94 %**	**99.94 %**	**99.94 %**	**99.94 %**
Maruthupandi et al. [11]	IADCL	99.70 %	99.72 %	99.70 %	99.71 %
	Immune Net	99.63 %	99.64 %	99.63 %	99.63 %
	XGB	99.09 %	99.03 %	99.07 %	99.07 %
	RF	98.81 %	98.82 %	98.81 %	98.81 %
	DT	98.54 %	93.41 %	96.76 %	95.06 %
	LR	92.96 %	90.80 %	90.96 %	90.87 %
Abdel-Basset et al. [12]	SS-Deep-ID	99.69 %	92.31 %	96.29 %	94.18 %
Tapu et al. [17]	Hybrid DL Model	95.68 %	96.50 %	95.68 %	96.10 %
Li et al. [18]	HDFEF	–	99.73 %	**99.96 %**	99.84 %
Yilmaz et al. [19]	RF	–	93.00 %	71.00 %	74.00 %
	CatBoost	–	89.00 %	83.00 %	83.00 %
	LightGBM	–	87.00 %	84.00 %	85.00 %
	XGBoost	–	89.00 %	85.00 %	86.00 %
Wang et al. [22]	GA-BPNN	98.00 %	98.50 %	95.00 %	96.50 %
Guo et al. [23]	TRBMA (BS-OSS)	99.88 %	99.86 %	99.88 %	99.89 %
Hariharan et al. [24]	Hybrid DL Model	99.00 %	97.00 %	96.00 %	97.00 %

We performed a satisfactory cleaning and adaptation of the selected dataset, according to the well-known selected strategy *Stratified 5-Fold CV*, selecting between all data rows, the 80% for the training subset and the remaining 20% as the test subset. The 80% for training has been used, applying the aforementioned technique, alternating, for each fold, a fifth of the data rows for the validation subset, basically 16% out of the total, and the remaining rows of data for training itself, which is 64%.

After all the *Fine-Tuning* processes, carried out in the experiments, we obtained models for a really different purpose from the initial model, more specifically for the detection of cyberattacks. All of them have been successfully built, since we obtained a very high success rate, very close to 100%, regardless of the dataset used. Despite this fact, there is still room for improvement as there is a type, *SQL Injection*, which does not classify as well as expected, due to the small amount of data available for this type of attack.

We compared our approach to LLM-based systems and other ML-based strategies, which used the same dataset. Our strategy outperforms other strategies specified in Section 4. Furthermore, we would like to emphasise the capacity for improvement of our model because we have selected the smallest size.

In future work, we have been thinking about different lines to follow and to improve this research.

The selection of values for the most hyperparameters has been carried out after studying the range of possible values and the impact on the output after carrying out *Fine-Tuning*, and the value for the maximum number of input tokens hyperparameter has been selected after applying a classic approach to optimise hyperparameters, training models, and studying results. One of the next steps is the use of specialised frameworks for hyperparameter optimisation, but keeping in mind all the possible hyperparameters.

We would like to improve the results even more after grouping the malignant classes by similarity.

We selected *Fine-Tuning* to perform our experiments. Our strategy achieved better results than other approaches mentioned in Section 4. However, studying the differences between the application of *Fine-Tuning* or *Transfer Learning* would be interesting. Therefore, we would like to apply the latter approach in order to check which one is better in our case.

Our line of research has been the design of a system for the detection of cyberattacks, after applying *Fine-Tuning* to an encoder-decoder pre-trained LLM for NLP tasks. We could find quite a lot of different approaches in the related works section, with their related results. Hence, we would like to apply a classic approach, for instance, a tailor-made model for the detection of cyberattacks, in order to compare which one is better.

The deployment module should be implemented to make use of the trained model in a real network. It is also important to validate the system with other network traffic models, e.g. with other public datasets.

Finally, given the promising results using the smallest size of the *T5* model, it would be very interesting to use other available sizes.

Author Contributions: Conceptualization, J.D. and I.M.; methodology, J.D.; software, L.G.; validation, L.G., J.D. and J.S.; formal analysis, L.G.; investigation, L.G., J.D. and I.M.; resources, I.M.; data curation, L.G.; writing—original draft preparation, L.G.; writing—review and editing, L.G., J.D. and I.M.; visualization, L.G. and J.S.; supervision, J.D. and I.M.; project administration, I.M.; funding acquisition, I.M. All authors have read and agreed to the published version of the manuscript.

Funding: This publication is part of the I+D+i grant PID2021-122215NB-C33 funded by MICIU/AEI/10.13039/501100011033 and ERDF/EU. This work was also partially supported by the Spanish Ministry of Science and Innovation under the AI4Software Spanish Research Network (Red2022-134647-T).

Data Availability Statement: The source code will be publicly available after this paper is published.

Acknowledgments: The authors thank the Systems Unit of the Information Systems Area of the University of Cádiz for computer resources and technical support.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A Validation with other datasets

Table A1 contains the confusion matrix for a model built with CIC-IDS-2017. 0.05% malignant rows, 42 out of 89,975, were classified as benign, and 0.01%, 11 out of 89,944, benign network flows were classified as malignant. Therefore, we could affirm that there is a negligible percentage of misclassification. We detected the worst results for the types of attacks 12 and 14, which are Infiltration and Heartbleed, respectively. For the first, the model only successfully classified 3 out of 7 network flows, which is 43%, and for the second, only 2 rows were classified as Infiltration. That means 100% of misclassification.

Table A2 contains the confusion matrix for the trained model with BCCC-CIC-IDS-2017. 0.07% malignant network flows, 49 out of 71,375, were classified as Benign, and 0.11% Benign rows of data, 75 out of 71,401, were classified as malignant. Therefore, there is a negligible percentage of misclassification using this dataset. We detected only one type of attack with negative results, more concretely the label 12, Web_SQL_Injection. In this case, 100% rows of data were misclassified, since 5 out of 5 network flows were classified as Benign.

Making a comparison between the confusion matrices for the models trained with CIC-IDS-2017 and its augmented dataset BCCC-CIC-IDS-2017, Tables A1 and A2, we found an interesting fact. The first model successfully classifies 100% SQL Injection data and misclassifies 100% of Heartbleed data. However, quite the opposite happens with the second model, since it misclassifies 100% SQL Injection rows of data and successfully classifies 100% Heartbleed data.

Table A1. Confusion Matrix, with y as rows and $\hat{y}$ as columns, for CIC-IDS-2017.

Label	BENIGN	DoS Hulk	PortScan	DDoS	DoS GoldenEye	FTP-Patator	SSH-Patator	DoS slowloris	DoS Slowhttptest	Bot	Web Attack - Brute Force	Web Attack - XSS	Infiltration	Web Attack - Sql Injection	Heartbleed	
Class	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	Misclassified
0	89,933	0	0	1	0	0	0	2	0	2	2	4	0	0	0	0.01 %
1	8	35,628	0	0	0	0	0	0	0	0	0	0	0	0	0	0.02 %
2	8	0	23,976	0	0	0	0	0	0	0	0	0	0	0	0	0.03 %
3	5	0	0	25,600	0	0	0	0	0	0	0	0	0	0	0	0.02 %
4	1	0	0	0	2,056	0	0	0	0	0	0	0	0	0	0	0.05 %
5	1	0	0	0	0	1,375	0	0	0	0	0	0	0	0	0	0.07 %
6	2	0	0	0	0	0	1,018	0	0	0	0	0	0	0	0	0.2 %
7	3	0	0	0	0	0	0	1,135	0	0	0	0	0	0	0	0.26 %
8	1	0	0	0	0	0	0	1	1,051	0	0	0	0	0	0	0.19 %
9	6	0	0	0	0	0	0	0	0	385	0	0	0	0	0	1.53 %
10	0	0	0	0	0	0	0	0	0	0	301	0	0	0	0	0 %
11	3	0	0	0	0	0	0	0	0	0	0	127	0	0	0	2.31 %
12	4	0	0	0	0	0	0	0	0	0	0	0	3	0	0	57.14 %
13	0	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0 %
14	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0	100 %

Table A2. Confusion Matrix, with y as rows and $\hat{y}$ as columns, for BCCC-CIC-IDS-2017.

Label	Benign	DoS_Hulk	Port_Scan	DDoS_LOIT	FTP-Patator	DoS_GoldenEye	DoS_Slowhttptest	SSH-Patator	Botnet_ARES	DoS_Slowloris	Web_Brute_Force	Web_XSS	Web_SQL_Injection	Heartbleed	
Class	0	1	2	3	4	5	6	7	8	9	10	11	12	13	Misclassified
0	71,326	13	4	4	14	0	1	4	3	1	25	6	0	0	0.11 %
1	0	69,240	0	0	0	0	0	0	0	0	0	0	0	0	0 %
2	6	0	32,259	0	0	0	0	0	0	0	0	0	0	0	0.02 %
3	1	0	0	19,145	0	0	0	0	0	0	0	0	0	0	0.01 %
4	3	0	0	0	1,903	0	0	0	0	0	0	0	0	0	0.16 %
5	5	0	0	0	0	1,667	0	0	0	1	0	0	0	0	0.36 %
6	11	0	0	0	0	0	1,359	0	0	1	0	0	0	0	0.88 %
7	0	0	0	0	0	0	0	1,190	0	0	0	0	0	0	0 %
8	0	0	0	0	0	0	0	0	1,102	0	0	0	0	0	0 %
9	7	0	0	0	0	0	3	0	0	1,015	0	0	0	0	0.98 %
10	10	0	0	0	0	0	0	0	0	0	537	0	0	0	1.83 %
11	1	0	0	0	0	0	0	0	0	0	0	270	0	0	0.37 %
12	5	0	0	0	0	0	0	0	0	0	0	0	0	0	100 %
13	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0 %

References

1. Admass, W.S.; Munaye, Y.Y.; Diro, A.A. Cyber security: State of the art, challenges and future directions, 2024. https://doi.org/10.1016/j.csa.2023.100031.
2. Vielberth, M.; Pernul, G. A Security Information and Event Management Pattern. In Proceedings of the 12th Latin American Conference on Pattern Languages of Programs (SLPLoP), November 2018.
3. Ashoor, A.S.; Gore, S. Difference between intrusion detection system (IDS) and intrusion prevention system (IPS). In Proceedings of the Advances in Network Security and Applications: 4th International Conference, CNSA 2011, Chennai, India, July 15-17, 2011 4. Springer, 2011, pp. 497–501.
4. Ashoor, A.S.; Gore, S. Importance of intrusion detection system (IDS). *International Journal of Scientific and Engineering Research* **2011**, *2*, 1–4.
5. Macas, M.; Wu, C.; Fuertes, W. A survey on deep learning for cybersecurity: Progress, challenges, and opportunities. *Computer Networks* **2022**, *212*. https://doi.org/10.1016/j.comnet.2022.109032.
6. LeCun, Y.; Bengio, Y.; Hinton, G. Deep Learning. *Nature* **2015**, *521*, 436–44. https://doi.org/10.1038/nature14539.
7. Burkov, A. *The Hundred-Page Machine Learning Book*; Andriy Burkov: Quebec City, Canada, 2019.
8. Church, K.W.; Chen, Z.; Ma, Y. Emerging trends: A gentle introduction to fine-tuning. *Natural Language Engineering* **2021**, *27*, 763–778. https://doi.org/10.1017/S1351324921000322.
9. Too, E.C.; Yujian, L.; Njuki, S.; Yingchun, L. A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture* **2019**, *161*, 272–279.
10. Alzubaidi, L.; Bai, J.; Al-Sabaawi, A.; Santamaría, J.; Albahri, A.; Al-dabbagh, B.S.N.; Fadhel, M.A.; Manoufali, M.; Zhang, J.; Al-Timemy, A.H.; et al. A survey on deep learning tools dealing with data scarcity: definitions, challenges, solutions, tips, and applications. *Journal of Big Data* **2023**, *10*, 46.
11. Maruthupandi, J.; Sivakumar, S.; Dhevi, B.L.; Prasanna, S.; Priya, R.K.; Selvarajan, S. An intelligent attention based deep convoluted learning (IADCL) model for smart healthcare security. *Scientific Reports* **2025**, *15*, 1363.

12. Abdel-Basset, M.; Hawash, H.; Chakrabortty, R.K.; Ryan, M.J. Semi-Supervised Spatiotemporal Deep Learning for Intrusions Detection in IoT Networks. *IEEE Internet of Things Journal* **2021**, *8*, 12251–12265. https://doi.org/10.1109/JIOT.2021.3060878.
13. Nie, L.; Wu, Y.; Wang, X.; Guo, L.; Wang, G.; Gao, X.; Li, S. Intrusion Detection for Secure Social Internet of Things Based on Collaborative Edge Computing: A Generative Adversarial Network-Based Approach. *IEEE Transactions on Computational Social Systems* **2022**, *9*, 134–145. https://doi.org/10.1109/TCSS.2021.3063538.
14. Kunang, Y.; Nurmaini, S.; Stiawan, D.; Suprapto, B. An end-to-end intrusion detection system with IoT dataset using deep learning with unsupervised feature extraction. *International Journal of Information Security* **2024**. https://doi.org/10.1007/s10207-023-00807-7.
15. Bhardwaj, S.; Dave, M. Enhanced neural network-based attack investigation framework for network forensics: Identification, detection, and analysis of the attack. *Computers & Security* **2023**, *135*, 103521. https://doi.org/10.1016/j.cose.2023.103521.
16. Wang, Z.; Ghaleb, F. An Attention-Based Convolutional Neural Network for Intrusion Detection Model. *IEEE Access* **2023**, *11*, 43116–43127. https://doi.org/10.1109/ACCESS.2023.3271408.
17. Tapu, S.; Shopnil, S.; Tamanna, R.; Dewan, M.; Alam, M. Malicious Data Classification in Packet Data Network Through Hybrid Meta Deep Learning. *IEEE Access* **2023**, *11*, 140609–140625. https://doi.org/10.1109/ACCESS.2023.3341911.
18. Li, Y.; Qin, T.; Huang, Y.; Lan, J.; Liang, Z.; Geng, T. HDFEF: A hierarchical and dynamic feature extraction framework for intrusion detection systems. *Computers & Security* **2022**, *121*, 102842. https://doi.org/10.1016/j.cose.2022.102842.
19. Yilmaz, M.; Bardak, B. An Explainable Anomaly Detection Benchmark of Gradient Boosting Algorithms for Network Intrusion Detection Systems. In Proceedings of the Proceedings - 2022 Innovations in Intelligent Systems and Applications Conference, ASYU 2022, 2022. https://doi.org/10.1109/ASYU56188.2022.9925451.
20. Hammad, M.; Hewahi, N.; Elmedany, W. MMM-RF: A novel high accuracy multinomial mixture model for network intrusion detection systems. *Computers and Security* **2022**, *120*. https://doi.org/10.1016/j.cose.2022.102777.
21. Yu, L.; Dong, J.; Chen, L.; Li, M.; Xu, B.; Li, Z.; Qiao, L.; Liu, L.; Zhao, B.; Zhang, C. PBCNN: Packet Bytes-based Convolutional Neural Network for Network Intrusion Detection. *Computer Networks* **2021**, *194*, 108117. https://doi.org/10.1016/j.comnet.2021.108117.
22. Wang, J.; Wang, X. An optimization model of computer network security based on GABP neural network algorithm. *EURASIP Journal on Information Security* **2025**, *2025*, 1–17.
23. Guo, D.; Xie, Y. Research on Network Intrusion Detection Model Based on Hybrid Sampling and Deep Learning. *Sensors* **2025**, *25*. https://doi.org/10.3390/s25051578.
24. Hariharan, S.; Annie Jerusha, Y.; Suganeshwari, G.; Syed Ibrahim, S.P.; Tupakula, U.; Varadharajan, V. A Hybrid Deep Learning Model for Network Intrusion Detection System Using Seq2Seq and ConvLSTM-Subnets. *IEEE Access* **2025**, *13*, 30705–30721. https://doi.org/10.1109/ACCESS.2025.3541399.
25. Liu, Y.; He, H.; Han, T.; Zhang, X.; Liu, M.; Tian, J.; Zhang, Y.; Wang, J.; Gao, X.; Zhong, T.; et al. Understanding LLMs: A comprehensive overview from training to inference. *Neurocomputing* **2025**, *620*, 129190. https://doi.org/https://doi.org/10.1016/j.neucom.2024.129190.
26. Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P.J. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *CoRR* **2020**, *1910.10683*, [1910.10683].
27. Li, F.; Shen, H.; Mai, J.; Wang, T.; Dai, Y.; Miao, X. Pre-trained language model-enhanced conditional generative adversarial networks for intrusion detection. *Peer-to-Peer Networking and Applications* **2024**, *17*, 227–245. https://doi.org/10.1007/s12083-023-01595-6.
28. Manocchio, L.D.; Layeghy, S.; Lo, W.W.; Kulatilleke, G.K.; Sarhan, M.; Portmann, M. FlowTransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications* **2024**, *241*. https://doi.org/10.1016/j.eswa.2023.122564.
29. Sokolova, M.; Lapalme, G. A systematic analysis of performance measures for classification tasks. *Information Processing and Management* **2009**, *45*, 427–437. https://doi.org/10.1016/j.ipm.2009.03.002.
30. UNSW. KDD Cup 1999 Data, 1999.
31. UNB. NSL-KDD dataset, 2009.
32. UNSW. The UNSW-NB15 Dataset, 2015.
33. UNB. Intrusion Detection Evaluation Dataset (CIC-IDS2017), 2017.
34. Sharafaldin, I.; Habibi Lashkari, A.; Ghorbani, A.A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the Proceedings of the 4th International Conference on Information Systems Security and Privacy - ICISSP. INSTICC, SciTePress, 2018, pp. 108–116. https://doi.org/10.5220/0006639801080116.
35. UNB. CICFlowMeter, 2016.
36. UNB. IPS/IDS dataset on AWS (CSE-CIC-IDS2018), 2018.
37. UNB. DDoS evaluation dataset (CIC-DDoS2019), 2019.

38. Sharafaldin, I.; Lashkari, A.H.; Hakak, S.; Ghorbani, A.A. Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. In Proceedings of the 2019 International Carnahan Conference on Security Technology (ICCST), 2019, pp. 1–8. https://doi.org/10.1109/CCST.2019.8888419.
39. of the Aegean, U. The AWID2 Dataset, 2016.
40. of the Aegean, U. The AWID3 Dataset, 2021.
41. Kolias, C.; Kambourakis, G.; Stavrou, A.; Gritzalis, S. Intrusion detection in 802.11 networks: Empirical evaluation of threats and a public dataset. *IEEE Communications Surveys and Tutorials* **2016**, *18*, 184–208. https://doi.org/10.1109/COMST.2015.2402161.
42. Chatzoglou, E.; Kambourakis, G.; Kolias, C. Empirical Evaluation of Attacks against IEEE 802.11 Enterprise Networks: The AWID3 Dataset. *IEEE Access* **2021**, *9*, 34188–34205. https://doi.org/10.1109/ACCESS.2021.3061609.
43. Shafi, M.; Lashkari, A.H.; Roudsari, A.H. NTLFlowLyzer: Towards generating an intrusion detection dataset and intruders behavior profiling through network and transport layers traffic analysis and pattern extraction. *Computers & Security* **2025**, *148*, 104160. https://doi.org/https://doi.org/10.1016/j.cose.2024.104160.
44. BCCC. Behaviour-Centric Cybersecurity Center (BCCC): Cybersecurity Datasets, 2025.
45. BCCC. NTLFlowLyzer, 2025.
46. Sharafaldin, I.; Lashkari, A.; Ghorbani, A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the Intl Conf. on Information Systems Security and Privacy (ICISSP), January 2018, Vol. 1, pp. 108–116. https://doi.org/https://doi.org/10.5220/0006639801080116.
47. Wu, Y.; Schuster, M.; Chen, Z.; Le, Q.V.; Norouzi, M.; Macherey, W.; Krikun, M.; Cao, Y.; Gao, Q.; Macherey, K.; et al. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. *CoRR* **2016**, *abs/1609.08144*, [1609.08144].
48. scikit-learn developers. Cross-validation: evaluating estimator performance, 2007–2024.
49. Roy, S. simpleT5, 2022.
50. Torre, D.; Mesadieu, F.; Chennamaneni, A. Deep learning techniques to detect cybersecurity attacks: a systematic mapping study. *Empirical Software Engineering* **2023**, *28*. https://doi.org/10.1007/s10664-023-10302-1.
51. Ferrag, M.A.; Ndhlovu, M.; Tihanyi, N.; Cordeiro, L.C.; Debbah, M.; Lestable, T.; Thandi, N.S. Revolutionizing Cyber Threat Detection with Large Language Models: A Privacy-Preserving BERT-Based Lightweight Model for IoT/IIoT Devices. *IEEE Access* **2024**, *12*, 23733–23750. https://doi.org/10.1109/ACCESS.2024.3363469.
52. G. Lira, O.; Marroquin, A.; To, M.A. Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems. In Proceedings of the Advanced Information Networking and Applications; Barolli, L., Ed., Cham, 2024; pp. 453–464.
53. Vo, H.; Du, H.; Nguyen, H. APELID: Enhancing real-time intrusion detection with augmented WGAN and parallel ensemble learning. *Computers and Security* **2024**, *136*. https://doi.org/10.1016/j.cose.2023.103567.
54. Amin, R.; El-Taweel, G.; Ali, A.; Tahoun, M. Hybrid Chaotic Zebra Optimization Algorithm and Long Short-Term Memory for Cyber Threats Detection. *IEEE Access* **2024**. https://doi.org/10.1109/ACCESS.2024.3397303.
55. Harrell, F.E. *Regression modeling strategies: with applications to linear models, logistic and ordinal regression, and survival analysis*, 2nd ed.; Springer: Cham, 2015.